Case Project: Computing machine

========== Machine 1,2,3,4,5 (Data type machine) ==========

General Directions:

- **Application platform:** Web-based application with a Graphical User Interface (GUI).
- **Programming languages:** Any programming language of your choice.
- **Application repository:** GitHub (must contain the source code and analysis write-up). Ensure the repository is set to public or that the instructor is granted access.

Required Outputs (All stored in the GitHub repository):

a.) **Screenshots:** Capture the program output for all possible test cases covering the specifications (normal cases, special cases, edge cases, different inputs, etc.).

b.) **Video Walkthrough:** A 5 to 8-minute video demonstrating the system on YouTube. Include the link in the GitHub `README.md`. Ensure both the GitHub repository and the YouTube video are accessible. The video must:

- Prove that the program is functioning correctly.
- Show all test cases covering the specifications (normal, special cases, different inputs, etc.).

c.) **Source Code:** Complete and well-commented source code.

d.) **Deployment Link:** Include the live website deployment link in the "About" / "Website" section of the GitHub repository.

e.) **Project Demo:** A live project demo may be required if needed. This can be conducted face-to-face or via Zoom.

Machine 1: Integer Machine

Process: Integer arithmetic and conversion.

1. Convert decimal to unsigned and signed binary

a. Input: A decimal number.

b. Input: Data size (ranging from 2 bits to 64 bits and beyond).

c. Output: Both unsigned and signed binary representations (must include error checking for out-of-bounds values).

2. Perform multiplication (sequential circuit binary multiplier), and division (non-restoring)

a. Input: Operands in either decimal (which must be converted to binary internally) or binary format.

b. Input: Data size (in bits).

c. Output: The step-by-step solution and the final result

Machine 2: Binary 32-bit Floating-Point Machine

Process: IEEE 754 binary single-precision operations.

1. Convert decimal to binary single-precision

a. Input: A decimal number.

b. Output: The IEEE 754 single-precision representation (including special cases like NaN, Infinity, etc.) in:

- i) Binary with proper spacing
- ii) Hexadecimal

2. Demonstrate rounding methods

a. Input: A number in either decimal or binary format.

b. Input: The target number of digits (or bits) for rounding.

c. Output: The rounded results using all four methods: chopping (truncation), round-up, round-down, and round-to-nearest ties-to-even.

3. Perform arithmetic operations (addition and multiplication) using rounding method

a. Input: Operands in either decimal or IEEE hexadecimal format.

b. Input: The type of operation (addition or multiplication).

c. Output: The step-by-step solution and final result (including special cases) in:

i) Binary with proper spacing.
ii) Hexadecimal.
iii) Decimal.

Machine 3: Binary 64-bit Floating-Point Machine
Process: IEEE 754 binary double-precision operations.

1. Convert decimal to binary double-precision
- **a. Input:** A decimal number.
- **b. Output:** The IEEE 754 double-precision representation (including special cases) in:
  - i) Binary with proper spacing.
  - ii) Hexadecimal.

2. Demonstrate rounding methods
- **a. Input:** A number in either decimal or binary format.
- **b. Input:** The target number of digits (or bits) for rounding.
- **c. Output:** The rounded results using all four methods: chopping, round-up, round-down, and round-to-nearest ties-to-even.

3. Perform arithmetic operations (addition and multiplication) using GRS method
- **a. Input:** Operands in either decimal or IEEE hexadecimal format.
- **b. Input:** The type of operation (addition or multiplication).
- **c. Output:** The step-by-step solution and final result (including special cases) in:
  - i) Binary with proper spacing.
  - ii) Hexadecimal.
  - iii) Decimal.

Machine 4: Decimal 32-bit Floating-Point Machine
Process: IEEE 754 decimal single-precision operations.

1. Convert decimal to decimal-based single-precision
- **a. Input:** A decimal number.
- **b. Output:** The IEEE 754 decimal single-precision representation (including special cases) in:
  - i) Binary with proper spacing.
  - ii) Hexadecimal.

2. Demonstrate rounding methods
- **a. Input:** A number in either decimal or binary format.
- **b. Input:** The target number of digits to be rounded.
- **c. Output:** The rounded results using all four methods: chopping, round-up, round-down, and round-to-nearest ties-to-even.

3. Perform arithmetic operations (subtraction and division) using rounding method
- **a. Input:** Operands in either decimal or IEEE hexadecimal format.
- **b. Input:** The type of operation (subtraction or division).
- **c. Output:** The step-by-step solution and final result (including special cases) in:
  - i) Decimal.
  - ii) Binary with proper spacing.
  - iii) Hexadecimal.

Machine 5: Decimal 64-bit Floating-Point Machine
Process: IEEE 754 decimal double-precision operations.

1. Convert decimal to decimal-based double-precision
- **a. Input:** A decimal number.
- **b. Output:** The IEEE 754 decimal double-precision representation (including special cases) in:
  - i) Binary with proper spacing.
  - ii) Hexadecimal.

2. Demonstrate rounding methods
- **a. Input:** A number in either decimal or binary format.
- **b. Input:** The target number of digits to be rounded.
- **c. Output:** The rounded results using all four methods: chopping, round-up, round-down, and round-to-nearest ties-to-even.

3. Perform arithmetic operations (subtraction and division) using GRS method
- **a. Input:** Operands in either decimal or IEEE hexadecimal format.
- **b. Input:** The type of operation (subtraction or division).
- **c. Output:** The step-by-step solution and final result (including special cases) in:

i) Decimal.
ii) Binary with proper spacing.
iii) Hexadecimal.

Machines 6, 7, 8, and 9: Cache Memory Machines
Common Specifications:

1. **Block Size:** Parameterized. Minimum of 2 words, no strict maximum (16 words or more is recommended). Must be a power of 2.
2. **Number of Cache Blocks:** Parameterized. Minimum of 4 blocks, no strict maximum (16 blocks or more is recommended). Must be a power of 2.
3. **Main Memory Size:** Fixed at 1024 blocks.
4. **Read Policy:** Parameterized (Non-load-through OR Load-through).

Test Cases *(Note: Let **n** be the total number of cache blocks)*:

a. Sequential sequence: Access up to 2n cache blocks. Repeat the sequence two times.
Example [if n = 4]: 0,1,2,3,4,5,6,7, 0,1,2,3,4,5,6,7

b. Mid-repeat blocks: Start at block 0 to n-1, then repeat the sequence up to 2n-1 twice. Afterward, reverse the sequence pattern.
Example [if n = 4]: 0,1,2,3, 0,1,2,3,4,5,6,7, 0,1,2,3,4,5,6,7, 3,2,1,0, 7,6,5,4,3,2,1,0, 7,6,5,4,3,2,1,0

c. Random sequence: Generate a random sequence of 64 block accesses (block indices must be within the 0 to 1023 range).

Required Outputs:
a. System Outputs:

- Visual snapshot of the cache memory state.
- A toggle/option to view either a step-by-step animated trace or just the final memory snapshot.
- A text log detailing the cache memory trace (required regardless of whether step-by-step or final snapshot is chosen).
- **Statistical Outputs:**
    1. Total memory access count; 2) Cache hit count; 3) Cache miss count; 4) Cache hit rate; 5) Cache miss rate; 6) Average Memory Access Time; 7) Total memory access time.

b. Analysis Write-up:

- A detailed analysis of the three test cases, including a comparison of the two cache operations for the specific machine.
- This must be submitted as a `README.md` file in the GitHub repository.
- *Note:* Ensure you specify the full specifications and parameters of your cache simulation system in the README.

c. Specific Machine Configurations:

Machine 6: Fully Associative (FA) + LRU **vs.** Fully Associative (FA) + MRU
Machine 7: 4-Way BSA + LRU **vs.** 4-Way BSA + MRU
Machine 8: Direct Mapped **vs.** Fully Associative (FA) + MRU
Machine 9: 8-Way BSA + LRU **vs.** 8-Way BSA + MRU